Exercícios Computacionais para a P2

Questões 1, 2 e 3 do livro texto estudado

Questão 1

Código

```
import itertools
import numpy as np

D = np.array([
    [0.105, 0.375],
    [0.180, 0.480],
    [0.000, 0.240]
])


def mva_exato(populacao, demandas):
    K, R = demandas.shape
    N = tuple(populacao)

    filas = {(0,) * R: np.zeros(K)}
    resultados = {}

    for total in range(1, sum(N) + 1):
        for estado in itertools.product(
            *[range(n + 1) for n in N]
        ):
            if sum(estado) != total:
                continue

            tempos = np.zeros((K, R))
            throughput = np.zeros(R)

            for r in range(R):
                if estado[r] == 0:
                    continue

                anterior = list(estado)
                anterior[r] -= 1
```

```
                n_observado = filas[tuple(anterior)]

                tempos[:, r] = (
                    demandas[:, r] * (1 + n_observado)
                )

                throughput[r] = (
                    estado[r] / tempos[:, r].sum()
                )

            n_centro = np.zeros(K)

            for r in range(R):
                n_centro += throughput[r] * tempos[:, r]

            filas[estado] = n_centro
            resultados[estado] = {
                "R_centro": tempos,
                "X": throughput,
                "n_centro": n_centro
            }

    return resultados


def mva_aproximado(populacao, demandas, tolerancia=0.001,
max_iter=10000):
    N = np.array(populacao, dtype=float)
    K, R = demandas.shape

    n = np.zeros((K, R))

    for r in range(R):
        centros = np.where(demandas[:, r] > 0)[0]
        if len(centros) > 0:
            n[centros, r] = N[r] / len(centros)

    for iteracao in range(1, max_iter + 1):
        n_anterior = n.copy()

        tempos = np.zeros((K, R))
        throughput = np.zeros(R)
```

```
        for r in range(R):
            if N[r] == 0:
                continue

            n_observado = np.zeros(K)

            for s in range(R):
                if N[s] == 0:
                    continue

                fator = (
                    N[s] - (1 if s == r else 0)
                ) / N[s]

                n_observado += n[:, s] * fator

            tempos[:, r] = (
                demandas[:, r] * (1 + n_observado)
            )

            throughput[r] = N[r] / tempos[:, r].sum()

        n = np.zeros((K, R))

        for r in range(R):
            n[:, r] = throughput[r] * tempos[:, r]

        erro = np.max(np.abs(n - n_anterior))

        if erro < tolerancia:
            break
    else:
        raise RuntimeError("O MVA aproximado não convergiu.")

    return {
        "iteracoes": iteracao,
        "X": throughput,
        "R": tempos.sum(axis=0),
        "n_classes": n.sum(axis=0),
        "n_centros": n.sum(axis=1),
        "R_centro": tempos,
        "erro_final": erro
    }
```

```python
def mostrar_exato(populacao, demandas):
    resultados = mva_exato(populacao, demandas)
    resultado = resultados[tuple(populacao)]

    print("População:", populacao)
    print("Throughput Q e U:", resultado["X"])
    print("Resposta Q e U:", resultado["R_centro"].sum(axis=0))
    print("Clientes médios por centro:", resultado["n_centro"])

    return resultado


def mostrar_aproximado(populacao, demandas):
    resultado = mva_aproximado(populacao, demandas)

    print("População:", populacao)
    print("Iterações:", resultado["iteracoes"])
    print("Throughput Q e U:", resultado["X"])
    print("Resposta Q e U:", resultado["R"])
    print("Clientes médios por classe:", resultado["n_classes"])
    print("Clientes médios por centro:", resultado["n_centros"])
    print("Erro final:", resultado["erro_final"])

    return resultado


print("=== MODELO ORIGINAL: MVA APROXIMADO ===")
aprox_original = mostrar_aproximado((3, 1), D)

print("\n=== MODELO ORIGINAL: MVA EXATO ===")
exato_original = mostrar_exato((3, 1), D)

print("\n=== ATUALIZAÇÕES TRIPLICADAS: MVA EXATO ===")
exato_triplicado = mostrar_exato((3, 3), D)

print("\n=== ATUALIZAÇÕES TRIPLICADAS: MVA APROXIMADO ===")
aprox_triplicado = mostrar_aproximado((3, 3), D)
```

Saída

```
=== MODELO ORIGINAL: MVA APROXIMADO ===
População: (3, 1)
Iterações: 10
Throughput Q e U: [3.99687758 0.40692307]
Resposta Q e U: [0.75058591 2.45746695]
Clientes médios por classe: [3. 1.]
Clientes médios por centro: [1.00296835 2.89937012 0.09766154]
Erro final: 0.0007163064577107026

=== MODELO ORIGINAL: MVA EXATO ===
População: (3, 1)
Throughput Q e U: [4.09388646 0.40933283]
Resposta Q e U: [0.7328     2.44299973]
Clientes médios por centro: [1.12162108 2.78013904 0.09823988]

=== ATUALIZAÇÕES TRIPLICADAS: MVA EXATO ===
População: (3, 3)
Throughput Q e U: [2.93809654 0.90644626]
Resposta Q e U: [1.02106924 3.30962809]
Clientes médios por centro: [1.55026165 4.19228347 0.25745488]

=== ATUALIZAÇÕES TRIPLICADAS: MVA APROXIMADO ===
População: (3, 3)
Iterações: 13
Throughput Q e U: [2.84614398 0.89889222]
Resposta Q e U: [1.05405771 3.33744128]
Clientes médios por classe: [3. 3.]
Clientes médios por centro: [1.35332031 4.39470273 0.25197696]
Erro final: 0.0006708831835040563
```

Comentário

Caso A — modelo original, população $(3, 1)$

O MVA exato reproduz, com pequenas diferenças de arredondamento, os resultados apresentados pelo livro: aproximadamente $4,093$ consultas por segundo e $0,409$ atualizações por segundo. O tempo de resposta das consultas é aproximadamente $0,733$ s, e o das atualizações, $2,443$ s.

O MVA aproximado converge em 10 iterações com a tolerância solicitada. Ele estima um tempo de resposta um pouco maior para ambas as classes e um throughput ligeiramente menor, ilustrando a diferença entre a solução exata e a aproximação.

Caso B — população de atualizações triplicada, $(3, 3)$

O aumento da população de atualizações de um para três clientes eleva a concorrência pela CPU e pelos discos. Consequentemente, o tempo médio de resposta das consultas aumenta de $0,7328$ s para $1,0211\ s$, aproximadamente 39,4%. Para as atualizações, o tempo aumenta de $2,4430$ s para $3,3096\ s$, aproximadamente 35,5%.

O throughput das atualizações cresce porque há três clientes dessa classe em vez de um. Entretanto, o throughput de consultas cai de aproximadamente $4,0939$ para $2,9381$ transações/s, devido à maior contenção dos recursos compartilhados.

O MVA exato calcula as métricas para cada vetor de população necessário à recursão. Com duas classes, o número de estados de população possíveis é $S = (N_Q + 1)(N_U + 1)$, assim, para $(3, 1)$: $S = 4 \times 2 = 8$ estados e para $(3, 3)$: $S = 4 \times 4 = 16$ estados.

Esses números incluem o estado inicial vazio. O MVA aproximado, por sua vez, executou 10 iterações para o modelo original e 13 para a população $(3, 3)$.

A contagem de estados e iterações é uma comparação prática do trabalho realizado, mas não equivale a uma contagem exata de operações de máquina: cada estado do MVA exato e cada iteração do aproximado executam quantidades diferentes de operações. Em geral, a vantagem do método aproximado cresce quando a população ou o número de classes torna a enumeração exata mais custosa.

Questão 2

Código

```
import numpy as np
from question1 import *

D_balanceado = np.array([
    [0.105, 0.375],
    [0.090, 0.480],
    [0.090, 0.240]
])

resultado_balanceado = mva_exato(
    (3, 1), D_balanceado
)[(3, 1)]

aprox_balanceado = mva_aproximado(
    (3, 1), D_balanceado, tolerancia=0.001
```

```
)

print("=== MVA EXATO: CARGA BALANCEADA ===")
print("Throughput Q e U:", resultado_balanceado["X"])
print("Resposta média Q e U:",
resultado_balanceado["R_centro"].sum(axis=0))
print("Clientes médios por centro:", resultado_balanceado["n_centro"])

print("\n=== MVA APROXIMADO: CARGA BALANCEADA ===")
print("Throughput Q e U:", aprox_balanceado["X"])
print("Resposta média Q e U:", aprox_balanceado["R"])
print("Iterações:", aprox_balanceado["iteracoes"])

original = mva_exato((3, 1), D)[(3, 1)]

X_original = original["X"]
R_original = original["R_centro"].sum(axis=0)

X_novo = resultado_balanceado["X"]
R_novo = resultado_balanceado["R_centro"].sum(axis=0)

print("\n=== VARIAÇÃO PERCENTUAL ===")

for r, classe in enumerate(["Q", "U"]):
    delta_X = (X_novo[r] / X_original[r] - 1) * 100
    delta_R = (R_novo[r] / R_original[r] - 1) * 100

    print(
        f"{classe}: throughput = {delta_X:+.2f}%; "
        f"tempo de resposta = {delta_R:+.2f}%"
    )
```

Saída

=== MODELO ORIGINAL: MVA APROXIMADO ===
População: (3, 1)
Iterações: 10
Throughput Q e U: [3.99687758 0.40692307]
Resposta Q e U: [0.75058591 2.45746695]
Clientes médios por classe: [3. 1.]
Clientes médios por centro: [1.00296835 2.89937012 0.09766154]
Erro final: 0.0007163064577107026

```
=== MODELO ORIGINAL: MVA EXATO ===
População: (3, 1)
Throughput Q e U: [4.09388646 0.40933283]
Resposta Q e U: [0.7328     2.44299973]
Clientes médios por centro: [1.12162108 2.78013904 0.09823988]

=== ATUALIZAÇÕES TRIPLICADAS: MVA EXATO ===
População: (3, 3)
Throughput Q e U: [2.93809654 0.90644626]
Resposta Q e U: [1.02106924 3.30962809]
Clientes médios por centro: [1.55026165 4.19228347 0.25745488]

=== ATUALIZAÇÕES TRIPLICADAS: MVA APROXIMADO ===
População: (3, 3)
Iterações: 13
Throughput Q e U: [2.84614398 0.89889222]
Resposta Q e U: [1.05405771 3.33744128]
Clientes médios por classe: [3. 3.]
Clientes médios por centro: [1.35332031 4.39470273 0.25197696]
Erro final: 0.0006708831835040563
=== MVA EXATO: CARGA BALANCEADA ===
Throughput Q e U: [5.24652949 0.45611889]
Resposta média Q e U: [0.57180656 2.19241082]
Clientes médios por centro: [1.5376982  1.41050164 1.05180016]

=== MVA APROXIMADO: CARGA BALANCEADA ===
Throughput Q e U: [5.24042201 0.45208702]
Resposta média Q e U: [0.57247298 2.21196354]
Iterações: 8

=== VARIAÇÃO PERCENTUAL ===
Q: throughput = +28.16%; tempo de resposta = -21.97%
U: throughput = +11.43%; tempo de resposta = -10.26%
```

Comentário

A modificação aumenta o throughput de consultas em aproximadamente 28,16% e reduz seu tempo de resposta médio em aproximadamente 21,97%. Para as atualizações, o throughput aumenta aproximadamente 11,43%, e o tempo de resposta diminui aproximadamente 10,26%.

O balanceamento beneficia as duas classes porque diminui a concentração de consultas no disco D1. No sistema original, esse disco tem a maior utilização total, de aproximadamente 93,34%

A demanda de consultas que antes se concentrava em D1 passa a consumir capacidade de D2, que estava pouco utilizado. O gargalo deixa de ser tão severo, e a utilização dos recursos de I/O fica mais equilibrada.

Essa análise supõe que a demanda total de consultas é dividida igualmente entre os discos e que os tempos de serviço podem ser representados pelos mesmos centros de serviço do modelo original.

Questão 3

Código

Item A

```
from line_solver import (
    Network,
    Source,
    Queue,
    Delay,
    Sink,
    OpenClass,
    ClosedClass,
    SchedStrategy,
    Exp,
    MVA
)


def construir_modelo(
    taxa_q=3.0,
    taxa_u=1.5,
    terminais=50,
    cpu_factor=1.0,
    d1_factor=1.0,
):

    model = Network("Exercicio 3 - Capitulo 6")

    source = Source(model, "Source")
    cpu = Queue(model, "CPU", SchedStrategy.PS)
    d1 = Queue(model, "D1", SchedStrategy.PS)
    d2 = Queue(model, "D2", SchedStrategy.PS)
    think = Delay(model, "Think")
```

```
sink = Sink(model, "Sink")

Q = OpenClass(model, "Consultas")
U = OpenClass(model, "Atualizacoes")

I = ClosedClass(model, "Interativos", terminais, think)

servicos = {
    cpu: {
        Q: 0.060 / cpu_factor,
        U: 0.100 / cpu_factor,
        I: 0.090 / cpu_factor,
    },
    d1: {
        Q: 0.030 / d1_factor,
        U: 0.030 / d1_factor,
        I: 0.045 / d1_factor,
    },
    d2: {
        Q: 0.060,
        U: 0.090,
    },
}

for centro, classes in servicos.items():
    for classe, demanda in classes.items():
        centro.set_service(classe, Exp(1.0 / demanda))

source.set_arrival(Q, Exp(taxa_q))
source.set_arrival(U, Exp(taxa_u))

think.set_service(I, Exp(1.0 / 15.0))

P = model.init_routing_matrix()

for classe in [Q, U]:
    P.add_route(classe, source, cpu, 1.0)
    P.add_route(classe, cpu, d1, 1.0)
    P.add_route(classe, d1, d2, 1.0)
    P.add_route(classe, d2, sink, 1.0)

P.add_route(I, think, cpu, 1.0)
P.add_route(I, cpu, d1, 1.0)
```

```
    P.add_route(I, d1, think, 1.0)

    model.link(P)

    return model


def resolver_e_exibir(**parametros):
    modelo = construir_modelo(**parametros)

    solver = MVA(modelo)
    resultados = solver.avg_table()

    print(resultados)

    return resultados


if __name__ == "__main__":
    resultados_base = resolver_e_exibir()
```

Item B

```
from item_a_modelo import *

resultados_aumento_q = resolver_e_exibir(
    taxa_q=5.85,
    taxa_u=1.5,
    terminais=50
)
```

Item C

```
from item_a_modelo import *

# Alternativa 1: substituir D1 por um disco duas vezes mais rápido
resultado_d1_rapido = resolver_e_exibir(
    taxa_q=5.85,
    taxa_u=1.5,
    terminais=50,
```

```
    d1_factor=2.0
)

# Alternativa 2: substituir a CPU por uma duas vezes mais rápida
resultado_cpu_rapida = resolver_e_exibir(
    taxa_q=5.85,
    taxa_u=1.5,
    terminais=50,
    cpu_factor=2.0
)
```

Item D

```
from item_a_modelo import *
import pandas as pd
import matplotlib.pyplot as plt

linhas = []

for M in range(50, 251, 10):
    resultados = resolver_e_exibir(
        taxa_q=5.85,
        taxa_u=1.5,
        terminais=M,
        cpu_factor=2.0
    )

    print(f"Terminais: {M} \n\n")

    linhas.append({
        "Terminais": M,
        "R_Q": resultados[resultados["JobClass"] ==
"Consultas"]["RespT"].sum(),
        "R_U": resultados[resultados["JobClass"] ==
"Atualizacoes"]["RespT"].sum(),
        "R_I": resultados[(resultados["JobClass"] == "Interativos") &
(resultados["Station"] != "Think")]["RespT"].sum()
    })

df = pd.DataFrame(linhas)

for coluna in ["R_Q", "R_U", "R_I"]:
```

```
    plt.plot(df["Terminais"], df[coluna], label=coluna)

plt.axhline(1.5, linestyle="--", label="Limite: 1,5 s")
plt.xlabel("Número de terminais")
plt.ylabel("Tempo médio de resposta (s)")
plt.title("Tempo de resposta versus número de terminais")
plt.legend()
plt.grid(True)
plt.show()
```

Saída

Item A

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.043955s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	3
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	0.47353	0.18	0.15784	0.15784	3	3
CPU	Atualizacoes	0.39461	0.15	0.26307	0.26307	1.5	1.5
CPU	Interativos	0.76257	0.2942	0.23328	0.23328	3.2689	3.2689
D1	Consultas	0.12526	0.09	0.041754	0.041754	3	3
D1	Atualizacoes	0.062631	0.045	0.041754	0.041754	1.5	1.5
D1	Interativos	0.20391	0.1471	0.06238	0.06238	3.2689	3.2689
D2	Consultas	0.26277	0.18	0.087591	0.087591	3	3
D2	Atualizacoes	0.19708	0.135	0.13139	0.13139	1.5	1.5
Think	Interativos	49.034	49.034	15	15	3.2689	3.2689

Item B

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.027992s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	5.85
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	1.63	0.351	0.27864	0.27864	5.85	5.85
CPU	Atualizacoes	0.6966	0.15	0.4644	0.4644	1.5	1.5
CPU	Interativos	1.3174	0.29073	0.40781	0.40781	3.2303	3.2303
D1	Consultas	0.27648	0.1755	0.047261	0.047261	5.85	5.85
D1	Atualizacoes	0.070891	0.045	0.047261	0.047261	1.5	1.5
D1	Interativos	0.22799	0.14536	0.07058	0.07058	3.2303	3.2303
D2	Consultas	0.68288	0.351	0.11673	0.11673	5.85	5.85
D2	Atualizacoes	0.26265	0.135	0.1751	0.1751	1.5	1.5

```
Think    Interativos     48.455   48.455        15        15  3.2303  3.2303
```

Item C

```
MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6]
completed in 0.024842s.
Station  JobClass         QLen     Util    RespT    ResidT    ArvR    Tput
Source   Consultas           0        0        0         0       0    5.85
Source   Atualizacoes        0        0        0         0       0     1.5
CPU      Consultas      1.6356    0.351  0.27958   0.27958    5.85    5.85
CPU      Atualizacoes  0.69896     0.15  0.46597   0.46597     1.5     1.5
CPU      Interativos    1.3252  0.29151  0.40913   0.40913  3.2391  3.2391
D1       Consultas     0.10741  0.08775  0.01836   0.01836    5.85    5.85
D1       Atualizacoes  0.02754   0.0225  0.01836   0.01836     1.5     1.5
D1       Interativos  0.089049 0.072879 0.027492  0.027492  3.2391  3.2391
D2       Consultas     0.68288    0.351  0.11673   0.11673    5.85    5.85
D2       Atualizacoes  0.26265    0.135   0.1751    0.1751     1.5     1.5
Think    Interativos    48.586   48.586        15        15  3.2391  3.2391
```

```
MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6]
completed in 0.019527s.
Station  JobClass         QLen     Util    RespT    ResidT    ArvR    Tput
Source   Consultas           0        0        0         0       0    5.85
Source   Atualizacoes        0        0        0         0       0     1.5
CPU      Consultas     0.29169   0.1755 0.049862  0.049862    5.85    5.85
CPU      Atualizacoes  0.12465    0.075 0.083103  0.083103     1.5     1.5
CPU      Interativos   0.24571  0.14856 0.074428  0.074428  3.3013  3.3013
D1       Consultas     0.27785   0.1755 0.047496  0.047496    5.85    5.85
D1       Atualizacoes 0.071244    0.045 0.047496  0.047496     1.5     1.5
D1       Interativos   0.23411  0.14856 0.070913  0.070913  3.3013  3.3013
D2       Consultas     0.68288    0.351 0.11673   0.11673    5.85    5.85
D2       Atualizacoes  0.26265    0.135  0.1751    0.1751     1.5     1.5
Think    Interativos     49.52    49.52        15        15  3.3013  3.3013
```

Item D

```
MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6]
completed in 0.071158s.
Station  JobClass         QLen     Util    RespT    ResidT    ArvR    Tput
Source   Consultas           0        0        0         0       0    5.85
Source   Atualizacoes        0        0        0         0       0     1.5
CPU      Consultas     0.29169   0.1755 0.049862  0.049862    5.85    5.85
CPU      Atualizacoes  0.12465    0.075 0.083103  0.083103     1.5     1.5
CPU      Interativos   0.24571  0.14856 0.074428  0.074428  3.3013  3.3013
D1       Consultas     0.27785   0.1755 0.047496  0.047496    5.85    5.85
```

D1 Atualizacoes 0.071244 0.045 0.047496 0.047496 1.5 1.5
D1 Interativos 0.23411 0.14856 0.070913 0.070913 3.3013 3.3013
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 49.52 49.52 15 15 3.3013 3.3013
Terminais: 50

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.018850s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.3067 0.1755 0.052427 0.052427 5.85 5.85
CPU Atualizacoes 0.13107 0.075 0.087378 0.087378 1.5 1.5
CPU Interativos 0.3098 0.17819 0.078238 0.078238 3.9597 3.9597
D1 Consultas 0.29144 0.1755 0.049819 0.049819 5.85 5.85
D1 Atualizacoes 0.074729 0.045 0.049819 0.049819 1.5 1.5
D1 Interativos 0.29446 0.17819 0.074365 0.074365 3.9597 3.9597
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 59.396 59.396 15 15 3.9597 3.9597
Terminais: 60

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.016361s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.3233 0.1755 0.055265 0.055265 5.85 5.85
CPU Atualizacoes 0.13816 0.075 0.092108 0.092108 1.5 1.5
CPU Interativos 0.3807 0.20778 0.082451 0.082451 4.6172 4.6172
D1 Consultas 0.3064 0.1755 0.052376 0.052376 5.85 5.85
D1 Atualizacoes 0.078564 0.045 0.052376 0.052376 1.5 1.5
D1 Interativos 0.3609 0.20778 0.078163 0.078163 4.6172 4.6172
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 69.258 69.258 15 15 4.6172 4.6172
Terminais: 70

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.022140s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.34175 0.1755 0.05842 0.05842 5.85 5.85

CPU Atualizacoes 0.14605 0.075 0.097366 0.097366 1.5 1.5
CPU Interativos 0.45951 0.23732 0.087132 0.087132 5.2737 5.2737
D1 Consultas 0.32293 0.1755 0.055202 0.055202 5.85 5.85
D1 Atualizacoes 0.082804 0.045 0.055202 0.055202 1.5 1.5
D1 Interativos 0.43434 0.23732 0.08236 0.08236 5.2737 5.2737
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 79.106 79.106 15 15 5.2737 5.2737
Terminais: 80

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.035191s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.36239 0.1755 0.061946 0.061946 5.85 5.85
CPU Atualizacoes 0.15487 0.075 0.10324 0.10324 1.5 1.5
CPU Interativos 0.54763 0.26681 0.092363 0.092363 5.9291 5.9291
D1 Consultas 0.34131 0.1755 0.058343 0.058343 5.85 5.85
D1 Atualizacoes 0.087515 0.045 0.058343 0.058343 1.5 1.5
D1 Interativos 0.51595 0.26681 0.08702 0.08702 5.9291 5.9291
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 88.936 88.936 15 15 5.9291 5.9291
Terminais: 90

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.027414s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.38559 0.1755 0.065913 0.065913 5.85 5.85
CPU Atualizacoes 0.16478 0.075 0.10986 0.10986 1.5 1.5
CPU Interativos 0.64673 0.29624 0.098242 0.098242 6.5831 6.5831
D1 Consultas 0.36183 0.1755 0.061852 0.061852 5.85 5.85
D1 Atualizacoes 0.092778 0.045 0.061852 0.061852 1.5 1.5
D1 Interativos 0.60712 0.29624 0.092224 0.092224 6.5831 6.5831
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 98.746 98.746 15 15 6.5831 6.5831
Terminais: 100

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.027130s.
Station JobClass QLen Util RespT ResidT ArvR Tput

Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.41187 0.1755 0.070405 0.070405 5.85 5.85
CPU Atualizacoes 0.17601 0.075 0.11734 0.11734 1.5 1.5
CPU Interativos 0.75895 0.32559 0.10489 0.10489 7.2354 7.2354
D1 Consultas 0.3849 0.1755 0.065795 0.065795 5.85 5.85
D1 Atualizacoes 0.098692 0.045 0.065795 0.065795 1.5 1.5
D1 Interativos 0.70956 0.32559 0.098068 0.098068 7.2354 7.2354
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 108.53 108.53 15 15 7.2354 7.2354
Terminais: 110

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.028751s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.44185 0.1755 0.07553 0.07553 5.85 5.85
CPU Atualizacoes 0.18882 0.075 0.12588 0.12588 1.5 1.5
CPU Interativos 0.88698 0.35486 0.11248 0.11248 7.8858 7.8858
D1 Consultas 0.41099 0.1755 0.070254 0.070254 5.85 5.85
D1 Atualizacoes 0.10538 0.045 0.070254 0.070254 1.5 1.5
D1 Interativos 0.82544 0.35486 0.10467 0.10467 7.8858 7.8858
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 118.29 118.29 15 15 7.8858 7.8858
Terminais: 120

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.029622s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.47633 0.1755 0.081424 0.081424 5.85 5.85
CPU Atualizacoes 0.20356 0.075 0.13571 0.13571 1.5 1.5
CPU Interativos 1.0342 0.38402 0.12119 0.12119 8.5339 8.5339
D1 Consultas 0.44071 0.1755 0.075334 0.075334 5.85 5.85
D1 Atualizacoes 0.113 0.045 0.075334 0.075334 1.5 1.5
D1 Interativos 0.95744 0.38402 0.11219 0.11219 8.5339 8.5339
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 128.01 128.01 15 15 8.5339 8.5339
Terminais: 130

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.017644s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	5.85
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	0.51637	0.1755	0.088268	0.088268	5.85	5.85
CPU	Atualizacoes	0.22067	0.075	0.14711	0.14711	1.5	1.5
CPU	Interativos	1.2052	0.41306	0.1313	0.1313	9.1791	9.1791
D1	Consultas	0.47483	0.1755	0.081167	0.081167	5.85	5.85
D1	Atualizacoes	0.12175	0.045	0.081167	0.081167	1.5	1.5
D1	Interativos	1.109	0.41306	0.12082	0.12082	9.1791	9.1791
D2	Consultas	0.68288	0.351	0.11673	0.11673	5.85	5.85
D2	Atualizacoes	0.26265	0.135	0.1751	0.1751	1.5	1.5
Think	Interativos	137.69	137.69	15	15	9.1791	9.1791

Terminais: 140

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.025713s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	5.85
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	0.56333	0.1755	0.096296	0.096296	5.85	5.85
CPU	Atualizacoes	0.24074	0.075	0.16049	0.16049	1.5	1.5
CPU	Interativos	1.4058	0.44193	0.14315	0.14315	9.8206	9.8206
D1	Consultas	0.51435	0.1755	0.087924	0.087924	5.85	5.85
D1	Atualizacoes	0.13189	0.045	0.087924	0.087924	1.5	1.5
D1	Interativos	1.2845	0.44193	0.1308	0.1308	9.8206	9.8206
D2	Consultas	0.68288	0.351	0.11673	0.11673	5.85	5.85
D2	Atualizacoes	0.26265	0.135	0.1751	0.1751	1.5	1.5
Think	Interativos	147.31	147.31	15	15	9.8206	9.8206

Terminais: 150

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.034421s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	5.85
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	0.61907	0.1755	0.10582	0.10582	5.85	5.85
CPU	Atualizacoes	0.26456	0.075	0.17637	0.17637	1.5	1.5
CPU	Interativos	1.6439	0.4706	0.15719	0.15719	10.458	10.458
D1	Consultas	0.56057	0.1755	0.095825	0.095825	5.85	5.85
D1	Atualizacoes	0.14374	0.045	0.095825	0.095825	1.5	1.5
D1	Interativos	1.4898	0.4706	0.14246	0.14246	10.458	10.458
D2	Consultas	0.68288	0.351	0.11673	0.11673	5.85	5.85
D2	Atualizacoes	0.26265	0.135	0.1751	0.1751	1.5	1.5
Think	Interativos	156.87	156.87	15	15	10.458	10.458

Terminais: 160

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.048146s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	5.85
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	0.68609	0.1755	0.11728	0.11728	5.85	5.85
CPU	Atualizacoes	0.2932	0.075	0.19547	0.19547	1.5	1.5
CPU	Interativos	1.9301	0.49901	0.17405	0.17405	11.089	11.089
D1	Consultas	0.6152	0.1755	0.10516	0.10516	5.85	5.85
D1	Atualizacoes	0.15774	0.045	0.10516	0.10516	1.5	1.5
D1	Interativos	1.7325	0.49901	0.15623	0.15623	11.089	11.089
D2	Consultas	0.68288	0.351	0.11673	0.11673	5.85	5.85
D2	Atualizacoes	0.26265	0.135	0.1751	0.1751	1.5	1.5
Think	Interativos	166.34	166.34	15	15	11.089	11.089

Terminais: 170

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.028926s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	5.85
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	0.76785	0.1755	0.13126	0.13126	5.85	5.85
CPU	Atualizacoes	0.32814	0.075	0.21876	0.21876	1.5	1.5
CPU	Interativos	2.2792	0.52709	0.19459	0.19459	11.713	11.713
D1	Consultas	0.6805	0.1755	0.11633	0.11633	5.85	5.85
D1	Atualizacoes	0.17449	0.045	0.11633	0.11633	1.5	1.5
D1	Interativos	2.0225	0.52709	0.17267	0.17267	11.713	11.713
D2	Consultas	0.68288	0.351	0.11673	0.11673	5.85	5.85
D2	Atualizacoes	0.26265	0.135	0.1751	0.1751	1.5	1.5
Think	Interativos	175.7	175.7	15	15	11.713	11.713

Terminais: 180

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.047627s.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR	Tput
Source	Consultas	0	0	0	0	0	5.85
Source	Atualizacoes	0	0	0	0	0	1.5
CPU	Consultas	0.86923	0.1755	0.14859	0.14859	5.85	5.85
CPU	Atualizacoes	0.37146	0.075	0.24764	0.24764	1.5	1.5
CPU	Interativos	2.7122	0.55474	0.22001	0.22001	12.328	12.328
D1	Consultas	0.75954	0.1755	0.12984	0.12984	5.85	5.85
D1	Atualizacoes	0.19475	0.045	0.12984	0.12984	1.5	1.5
D1	Interativos	2.3736	0.55474	0.19254	0.19254	12.328	12.328

D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 184.91 184.91 15 15 12.328 12.328
Terminais: 190

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.046698s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 0.99716 0.1755 0.17046 0.17046 5.85 5.85
CPU Atualizacoes 0.42614 0.075 0.28409 0.28409 1.5 1.5
CPU Interativos 3.2585 0.58181 0.25203 0.25203 12.929 12.929
D1 Consultas 0.85645 0.1755 0.1464 0.1464 5.85 5.85
D1 Atualizacoes 0.2196 0.045 0.1464 0.1464 1.5 1.5
D1 Interativos 2.804 0.58181 0.21687 0.21687 12.929 12.929
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 193.94 193.94 15 15 12.929 12.929
Terminais: 200

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.036799s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 1.1617 0.1755 0.19858 0.19858 5.85 5.85
CPU Atualizacoes 0.49646 0.075 0.33097 0.33097 1.5 1.5
CPU Interativos 3.9613 0.6081 0.29314 0.29314 13.513 13.513
D1 Consultas 0.97679 0.1755 0.16697 0.16697 5.85 5.85
D1 Atualizacoes 0.25046 0.045 0.16697 0.16697 1.5 1.5
D1 Interativos 3.3385 0.6081 0.24705 0.24705 13.513 13.513
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 202.7 202.7 15 15 13.513 13.513
Terminais: 210

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.033449s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 1.3775 0.1755 0.23547 0.23547 5.85 5.85
CPU Atualizacoes 0.58867 0.075 0.39244 0.39244 1.5 1.5
CPU Interativos 4.8827 0.63332 0.34694 0.34694 14.074 14.074

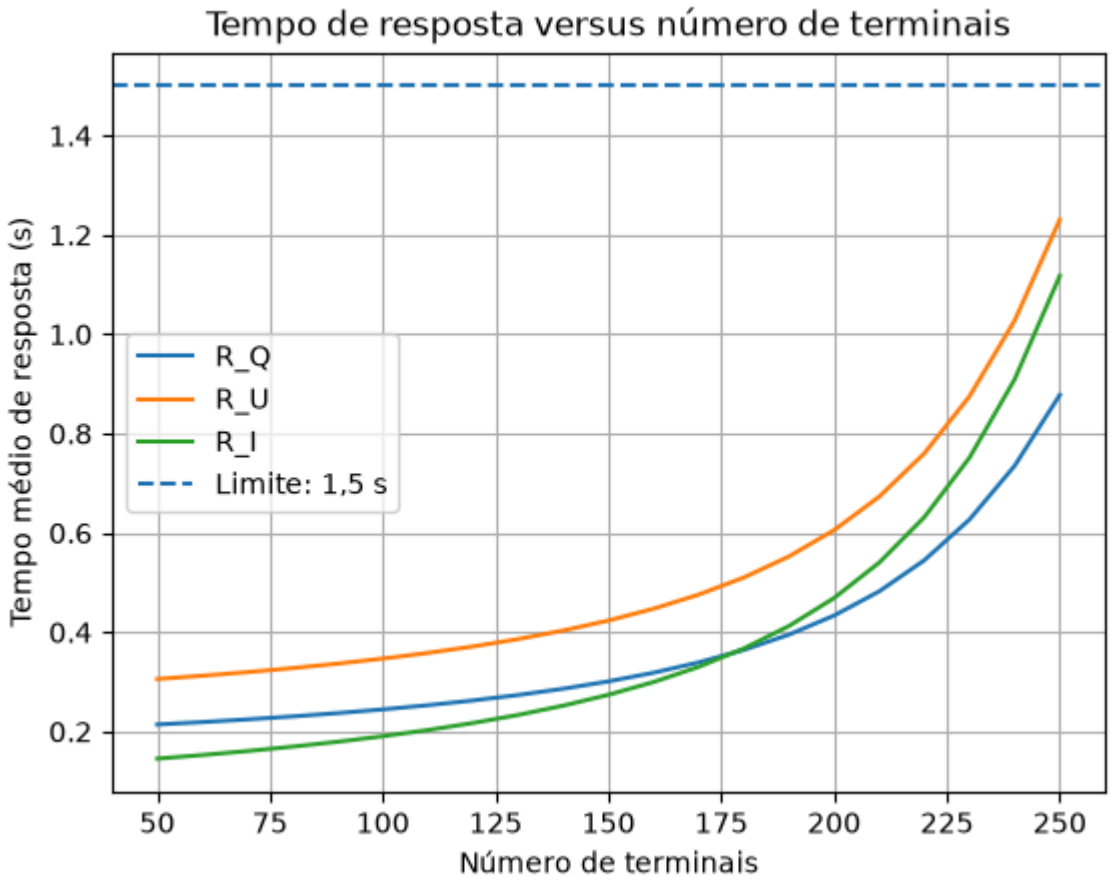
D1 Consultas 1.128 0.1755 0.19282 0.19282 5.85 5.85
D1 Atualizacoes 0.28922 0.045 0.19282 0.19282 1.5 1.5
D1 Interativos 4.01 0.63332 0.28493 0.28493 14.074 14.074
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 211.11 211.11 15 15 14.074 14.074
Terminais: 220

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.038038s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 1.6655 0.1755 0.28471 0.28471 5.85 5.85
CPU Atualizacoes 0.71177 0.075 0.47451 0.47451 1.5 1.5
CPU Interativos 6.113 0.65708 0.41864 0.41864 14.602 14.602
D1 Consultas 1.3194 0.1755 0.22555 0.22555 5.85 5.85
D1 Atualizacoes 0.33832 0.045 0.22555 0.22555 1.5 1.5
D1 Interativos 4.8604 0.65708 0.33287 0.33287 14.602 14.602
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 219.03 219.03 15 15 14.602 14.602
Terminais: 230

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.067127s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5
CPU Consultas 2.0554 0.1755 0.35136 0.35136 5.85 5.85
CPU Atualizacoes 0.87839 0.075 0.58559 0.58559 1.5 1.5
CPU Interativos 7.7781 0.67885 0.5156 0.5156 15.086 15.086
D1 Consultas 1.5622 0.1755 0.26704 0.26704 5.85 5.85
D1 Atualizacoes 0.40056 0.045 0.26704 0.26704 1.5 1.5
D1 Interativos 5.9387 0.67885 0.39367 0.39367 15.086 15.086
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 226.28 226.28 15 15 15.086 15.086
Terminais: 240

MVA analysis [method: default/exact; type: exact, deterministic; lang: python; env: 3.14.6] completed in 0.044912s.
Station JobClass QLen Util RespT ResidT ArvR Tput
Source Consultas 0 0 0 0 0 5.85
Source Atualizacoes 0 0 0 0 0 1.5

CPU Consultas 2.5859 0.1755 0.44203 0.44203 5.85 5.85
CPU Atualizacoes 1.1051 0.075 0.73672 0.73672 1.5 1.5
CPU Interativos 10.043 0.698 0.6475 0.6475 15.511 15.511
D1 Consultas 1.8668 0.1755 0.3191 0.3191 5.85 5.85
D1 Atualizacoes 0.47865 0.045 0.3191 0.3191 1.5 1.5
D1 Interativos 7.2914 0.698 0.47008 0.47008 15.511 15.511
D2 Consultas 0.68288 0.351 0.11673 0.11673 5.85 5.85
D2 Atualizacoes 0.26265 0.135 0.1751 0.1751 1.5 1.5
Think Interativos 232.67 232.67 15 15 15.511 15.511
Terminais: 250



Comentário

Item A: Modelo base

O sistema apresenta utilização significativa da CPU, principalmente devido às classes Q, U e I. O tempo de resposta da classe interativa I é de aproximadamente 0,296 s, com vazão de 3,269 transações/s.

Item B: Aumento de 95% na taxa de chegada de Q

O aumento da carga de Q eleva os tempos de resposta de todas as classes devido à maior disputa pelos recursos. A resposta de I aumenta para 0,478 s, enquanto sua vazão diminui para 3,230 transações/s.

Item C: Comparação das melhorias

A redução pela metade do tempo de serviço da CPU apresenta resultados melhores do que acelerar o dispositivo D1. A CPU é o recurso cuja melhoria proporciona a maior redução nos tempos de resposta e o maior aumento na vazão de I.

Item D: Variação do número de terminais

Ao aumentar o número de terminais interativos, os tempos de resposta das três classes crescem devido à maior concorrência pelos recursos. Entretanto, com a CPU mais rápida, a resposta de I permanece abaixo de 1,5 s até 250 terminais, que é o maior valor testado.